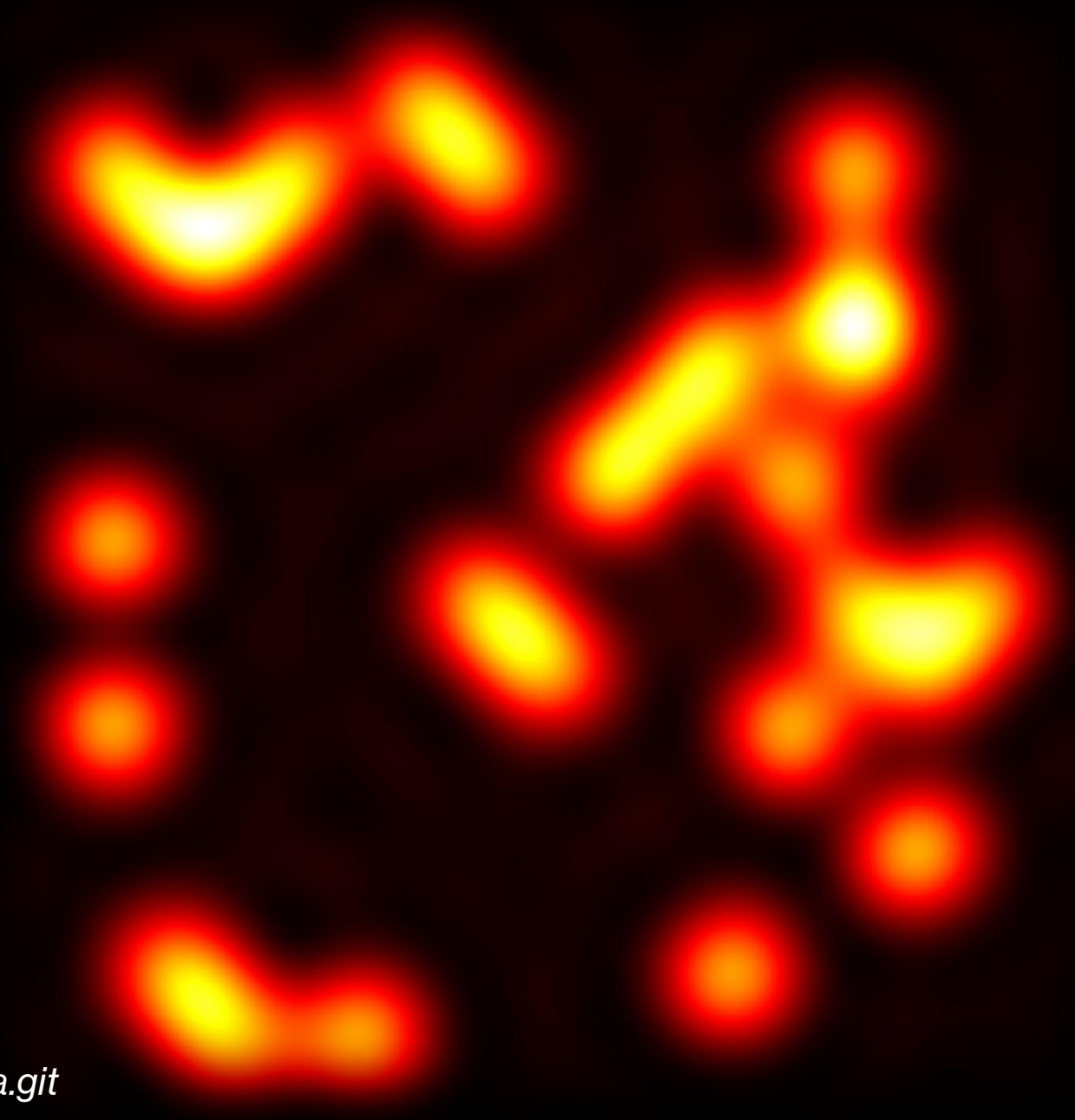
Stochastic Super Resolution

A classical Monte-Carlo simulation of imaging atoms in a 2D-lattice

AuroraStudent:
Luigi Petillo

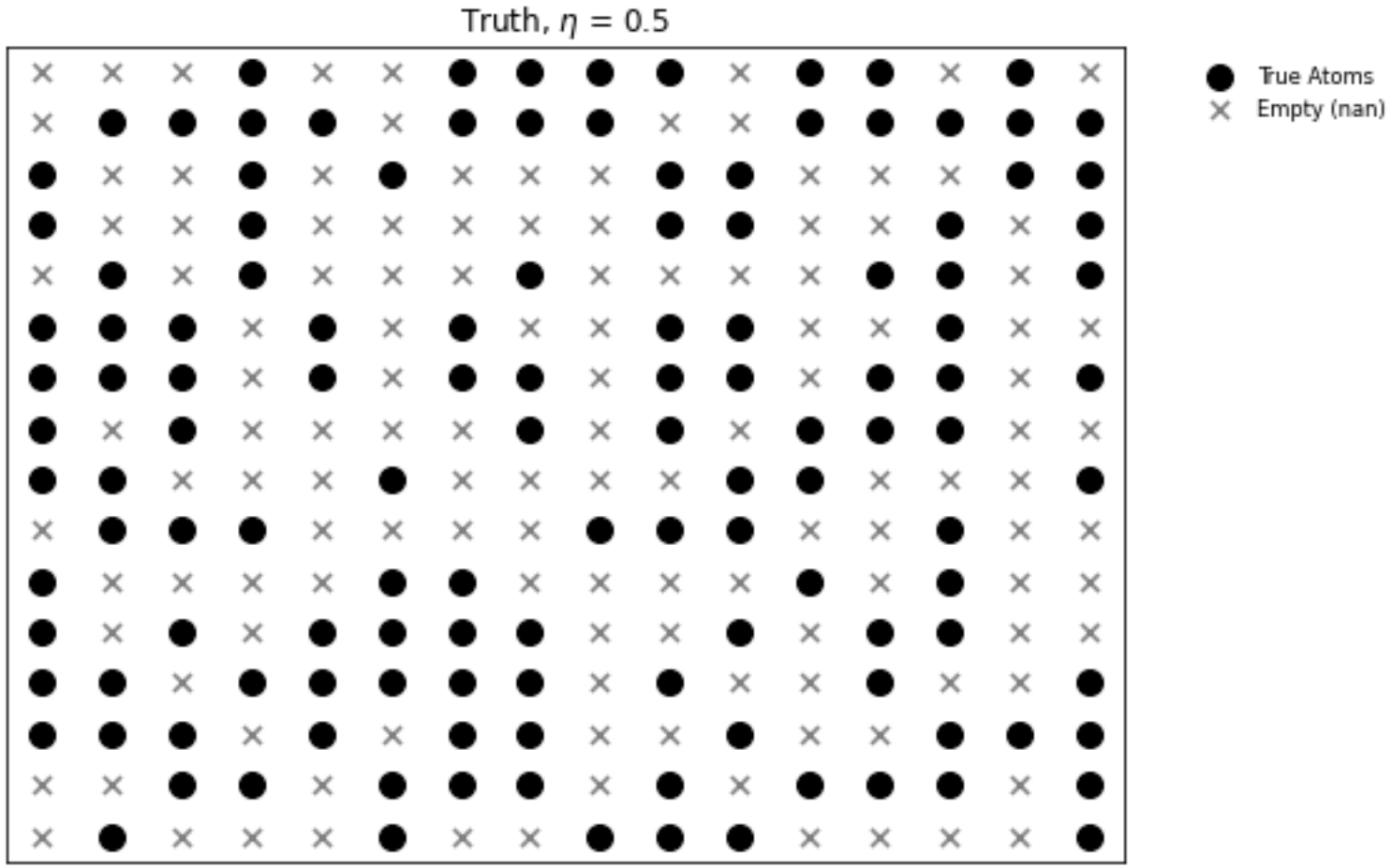
Supervisors:
Arfor Houwman,
Francesca Ferlaino

https://github.com/Arfor/StochasticSuperResolutionAurora.git



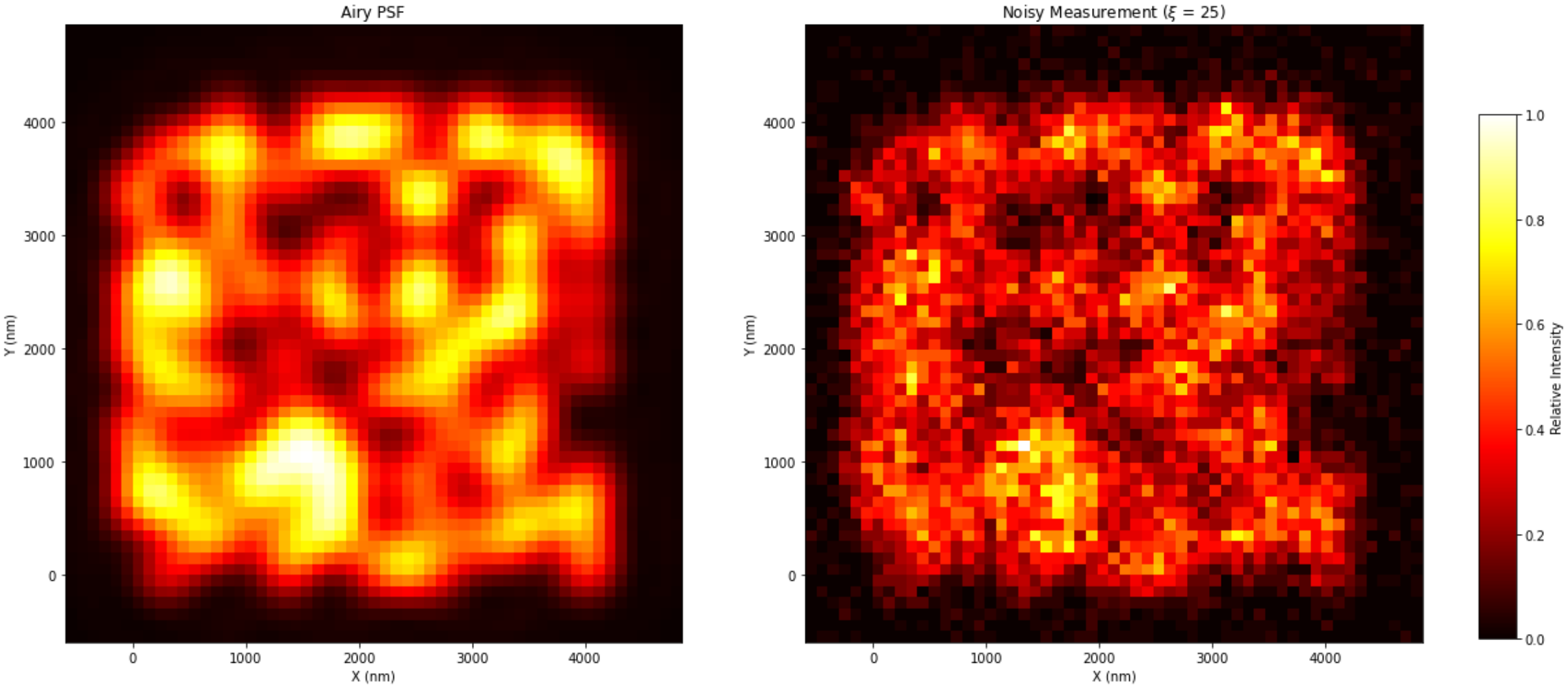
Lattice and images properties

- 16 x 16 sites
- x-spacing: 266nm
- y-spacing: 266nm
- filling fraction $\boldsymbol{\eta}$
- unshelving fraction q, to be updated for each image
- Poissonian noise-level ξ (lower ξ = lower noise)
- $\boldsymbol{\lambda_{shelving}} = 1299\mathrm{nm}$
- $\boldsymbol{\lambda_{imaging}} = 401\mathrm{nm}$
- $N.A. = 0.43$
- magnification: 100nm per pixel (possible for the experiments, but hard...)

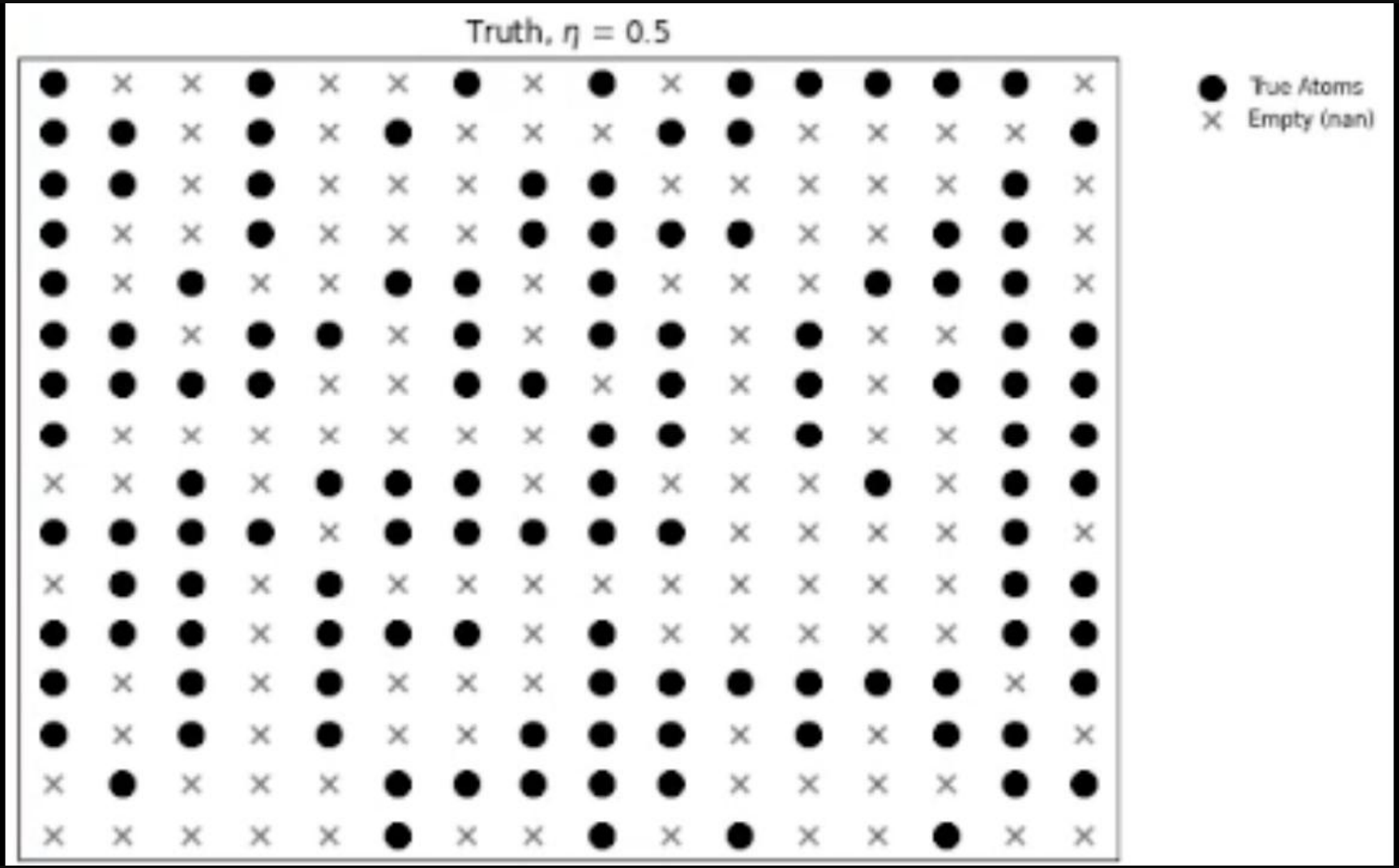


What happens if we image the lattice?...

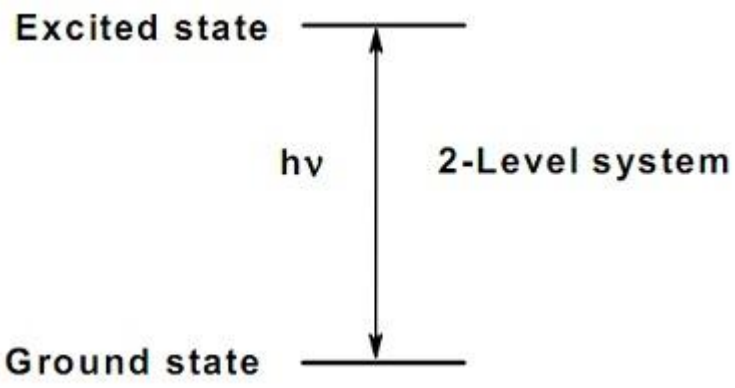
Here's the problem:
Where are the atoms? How many atoms are there?



Our solution: the Imaging Process

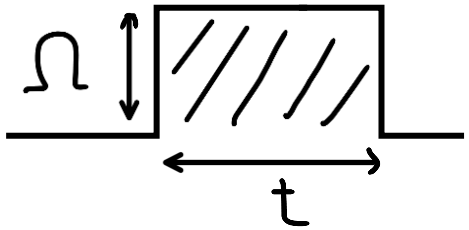


Unshelving fraction q



$$\| \psi > = \cos\left(\frac{\Omega t}{2}\right) |E> \; + \sin\left(\frac{\Omega t}{2}\right) \; |G>$$

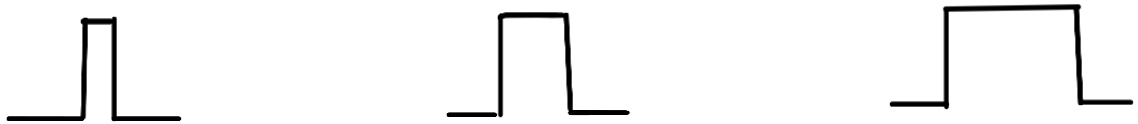
Ω= Rabi frequency



(Laser pulse)

$$\boldsymbol{q} = sin^2\left(\frac{\Omega t}{2}\right)$$

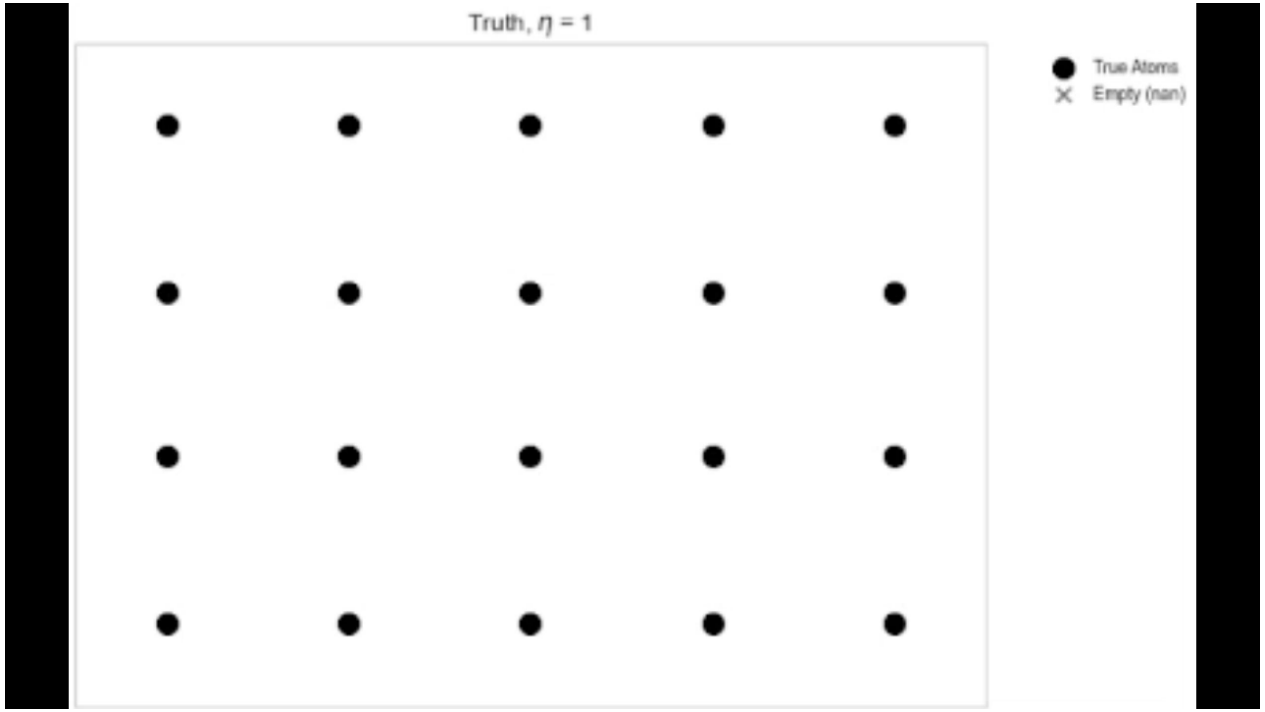
We want the same number of atoms for each image, that's why we expect that we need to increase q

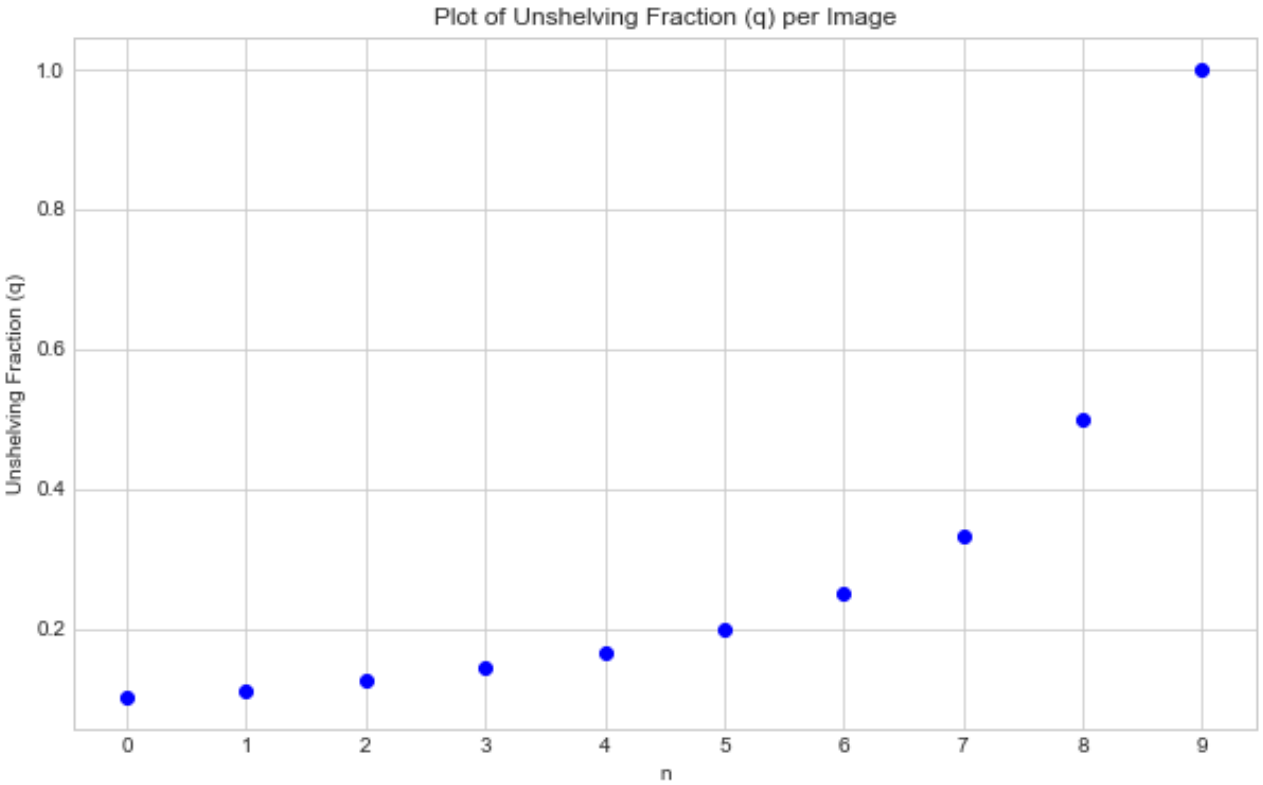


Example: 1st 2nd 3rd

Let's see it more quantitatively...

How to increase the unshelving fraction?





Example: $\mathrm{N_{img}} = 10$, $\mathrm{N_{atoms}} = \mathrm{N_0} = 20, q_0 = \frac{1}{N_{img}} = \frac{1}{10}$

How to increase the unshelving fraction?

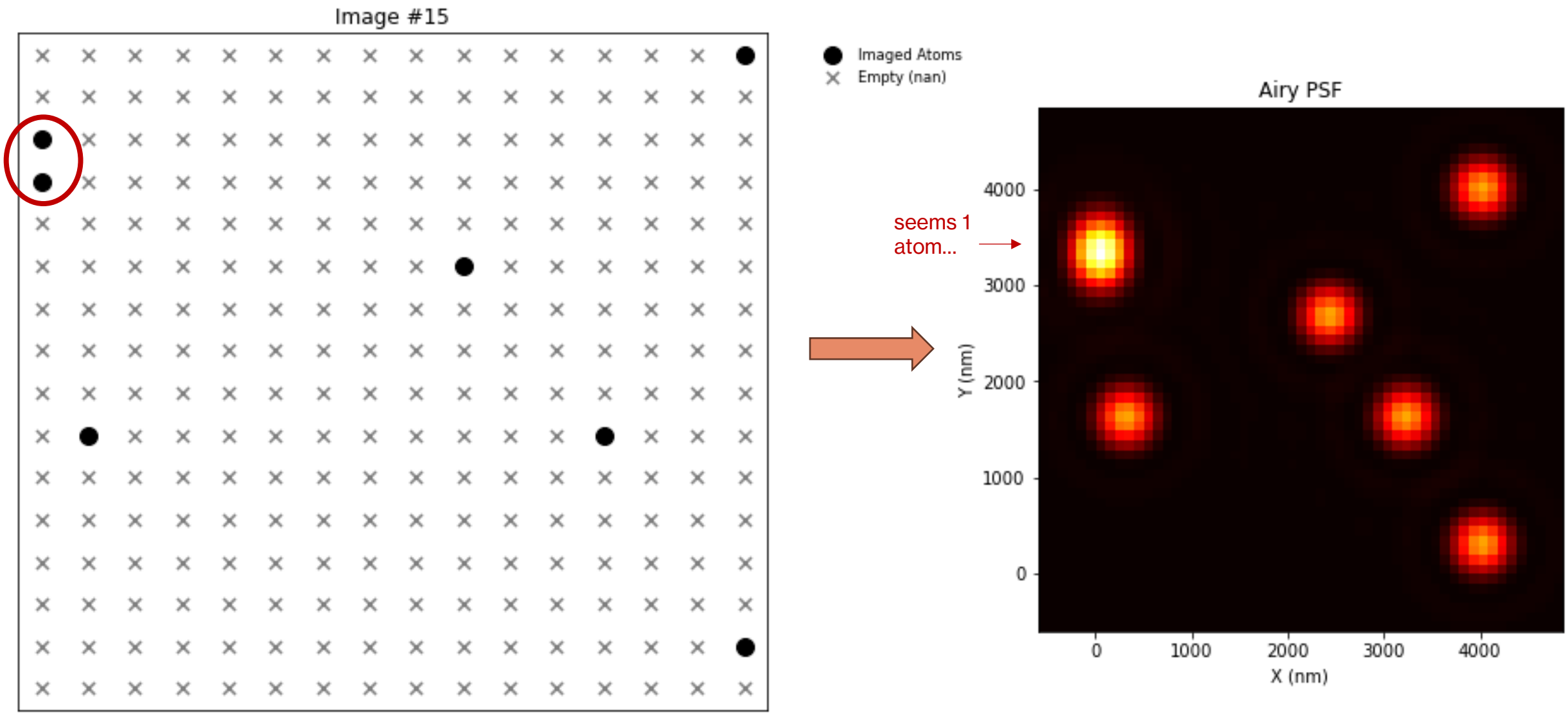
1. Fix $\mathrm{N_{img}}$ and $\mathrm{N_{atoms}} = \mathrm{N_0} = \eta \cdot 16 \cdot 16$
2. Shelve every atom and choose $q_0 = \frac{1}{N_{img}}$
3. Unshelve $\mathrm{N_0} \cdot q_0$ atoms and image
4. Because we want the same number of atoms for each image, update $q_1 = \frac{\mathrm{q_0}}{1-\mathrm{q_0}}$, unshelve and image $\mathrm{N_1} \cdot q_1 = \mathrm{N_0} \cdot q_0$, where $\mathrm{N_1} = \mathrm{N_0}(1 - \mathrm{q_0})$ are the remaining atoms
5. Update $\boldsymbol{q_n} = \frac{\boldsymbol{q_0}}{\boldsymbol{1-n \cdot q_0}}$ for each step (iteratively)
6. We reach $q_n = 1$ when $\mathrm{n} = \frac{1}{\mathrm{q_0}} - 1 = N_{img} - 1$, and image the remaining atoms

Example:

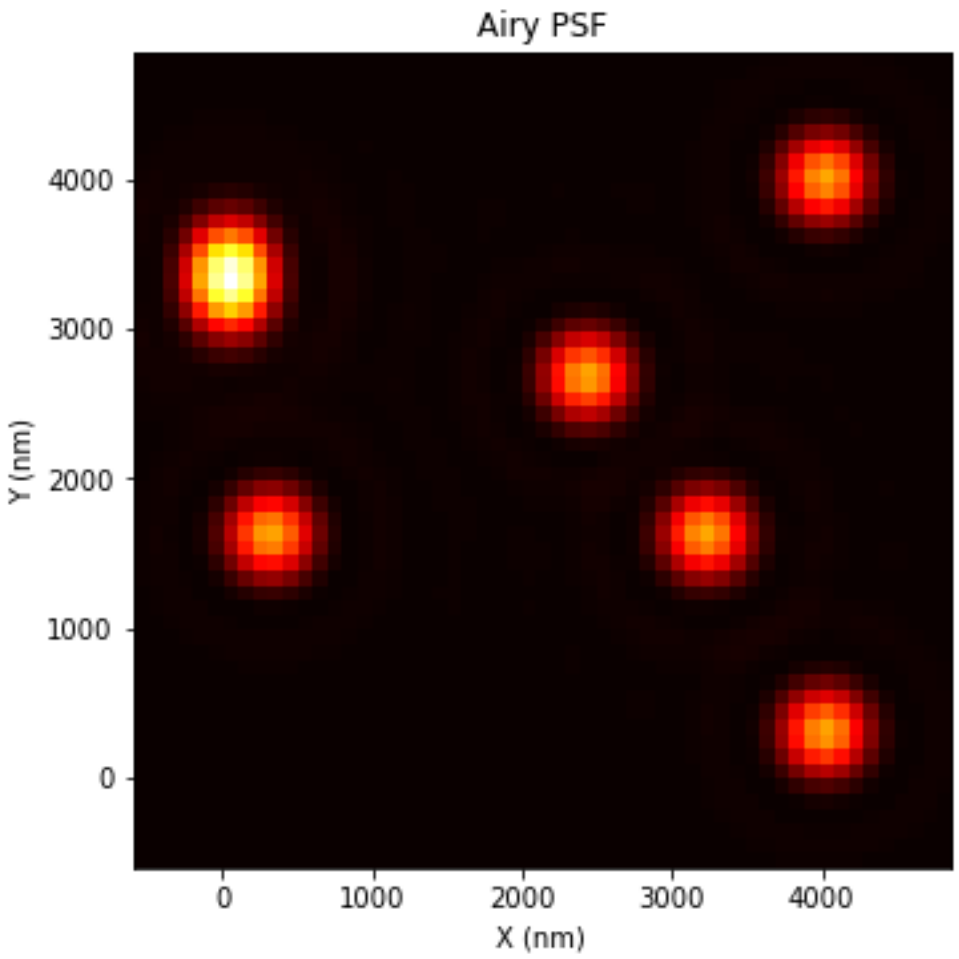
1. $\mathrm{N_{img}} = 10$, $\mathrm{N_{atoms}} = \mathrm{N_0} = 20$
2. $\mathrm{q_0} = \frac{1}{10}$
3. image 2 atoms among 20
4. $\mathrm{q_1} = \frac{1}{9}$, unshelve and image 18/9= 2 atoms again
5. $\mathrm{q_2} = \frac{1}{8}$, unshelve and image 16/8= 2 atoms again; $\mathrm{q_3} = \frac{1}{7}$, unshelve and image 14/7= 2 atoms again

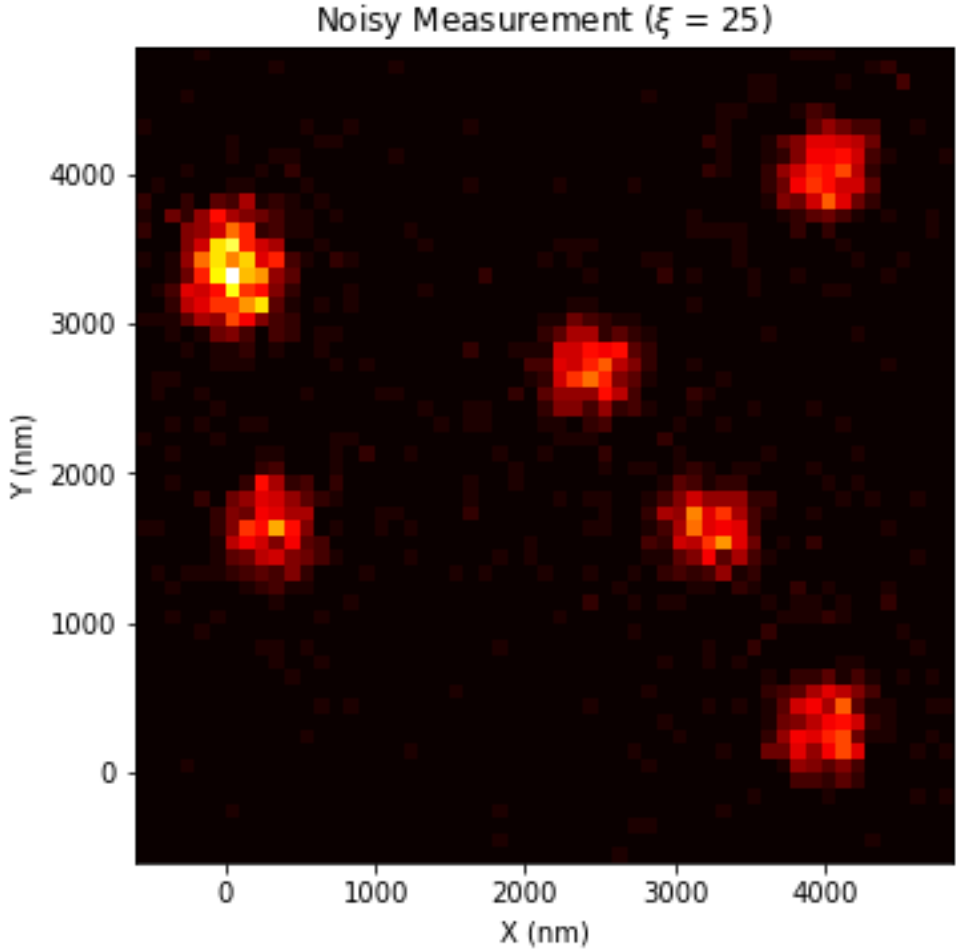
etc etc

Image-Convolution with PSF



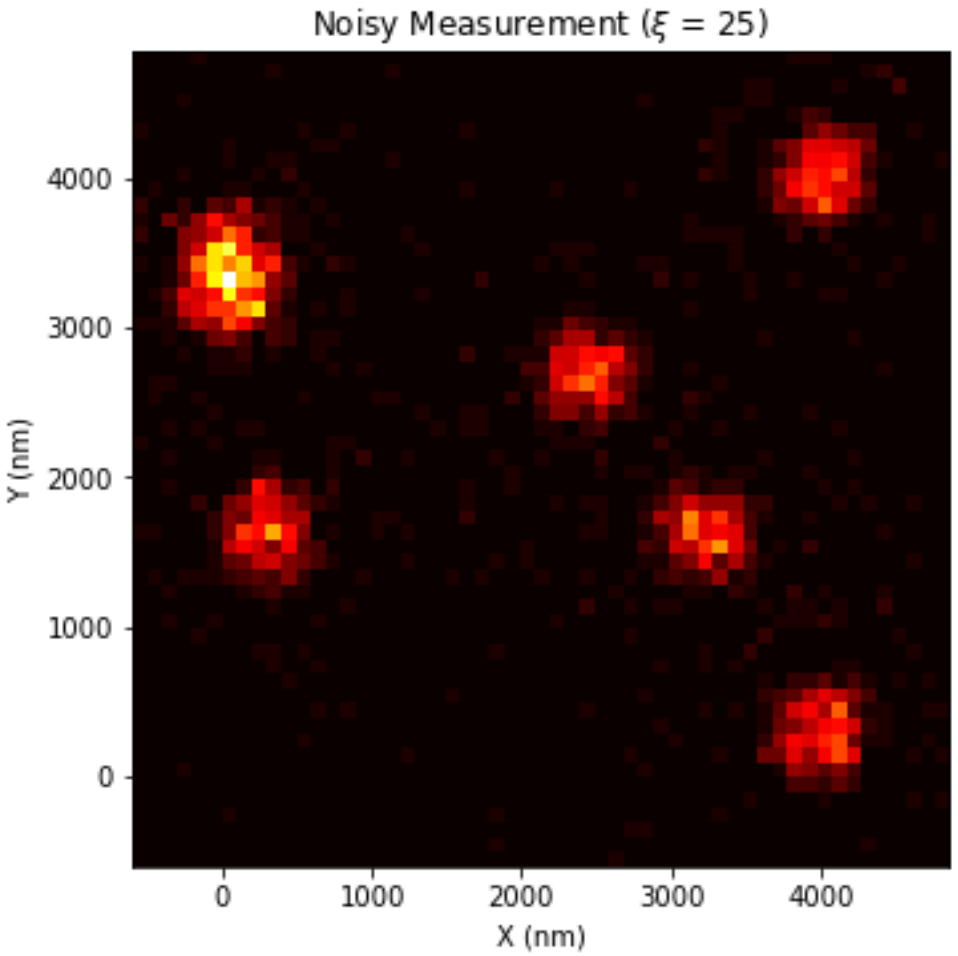
Adding Poissonian Noise

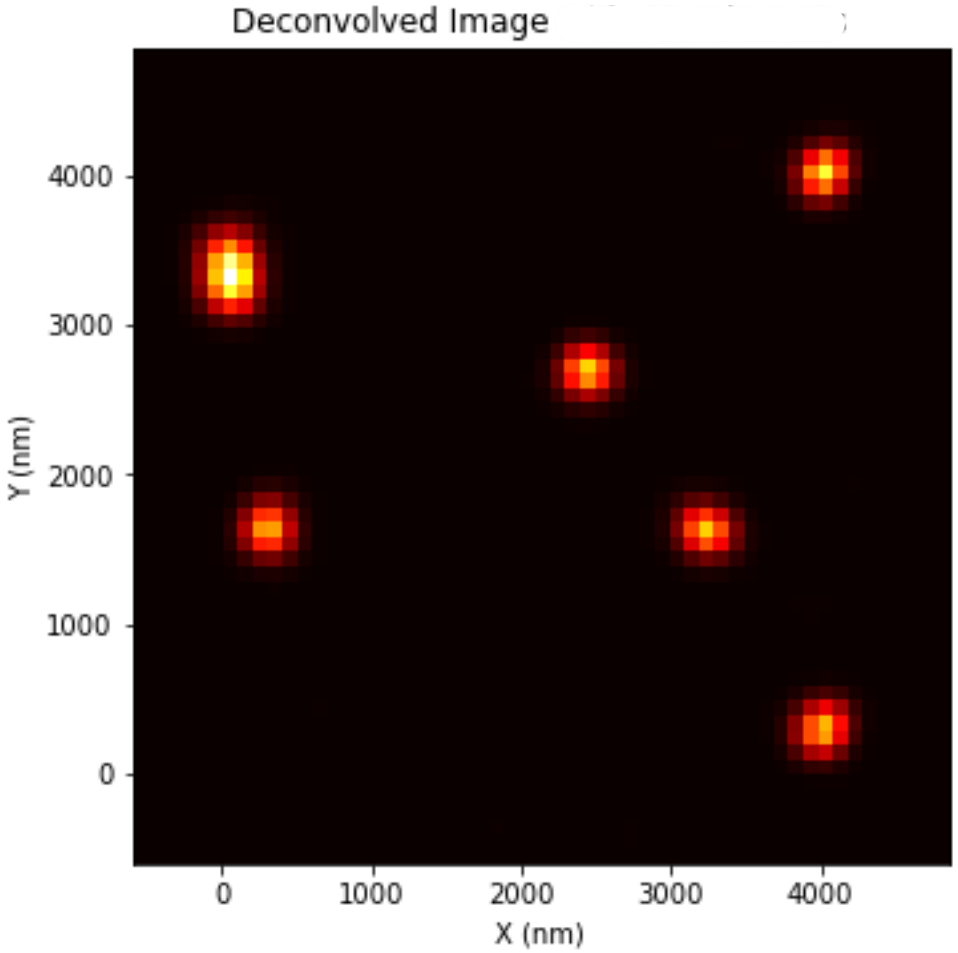




This is what we have from experiments!
How to reverse this process?! (next slides!)

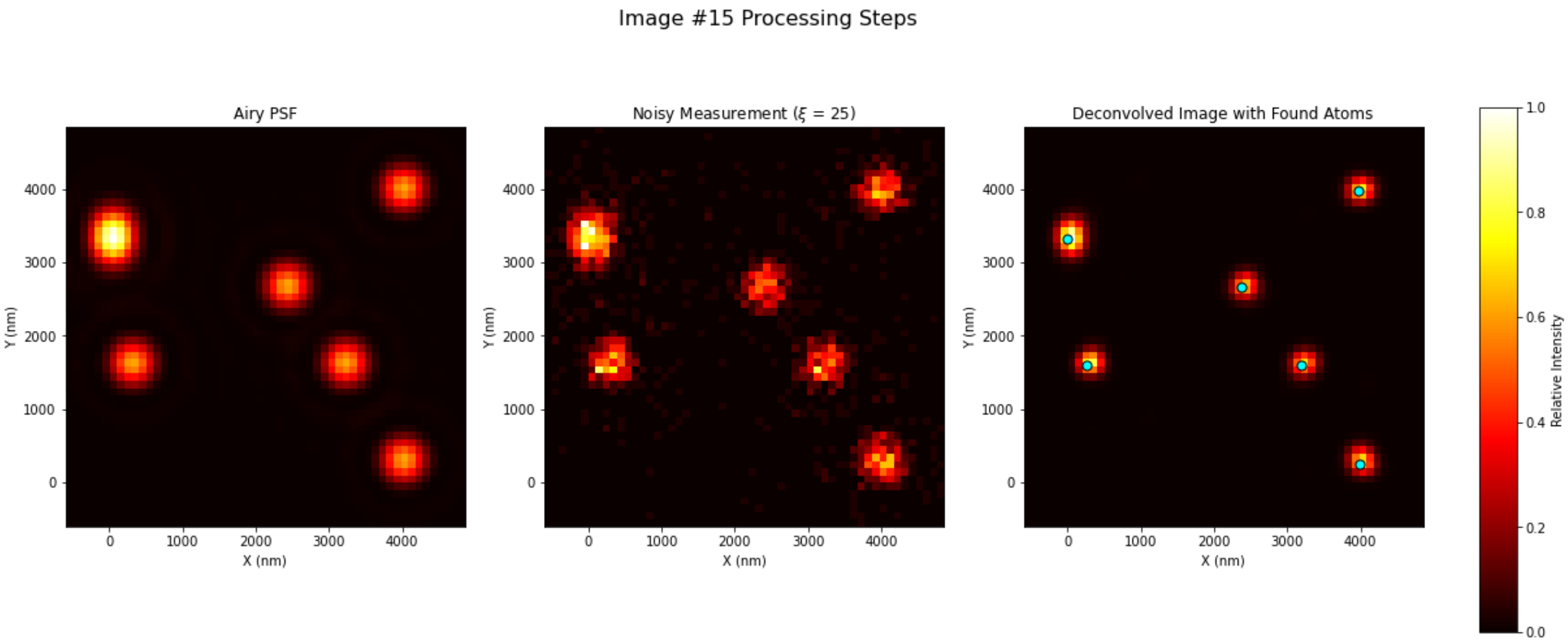
Richardson-Lucy Deconvolution





Here I chose 7 iterations for RL-algorithm, as suggested in the article, where you can find details and other possible methods:

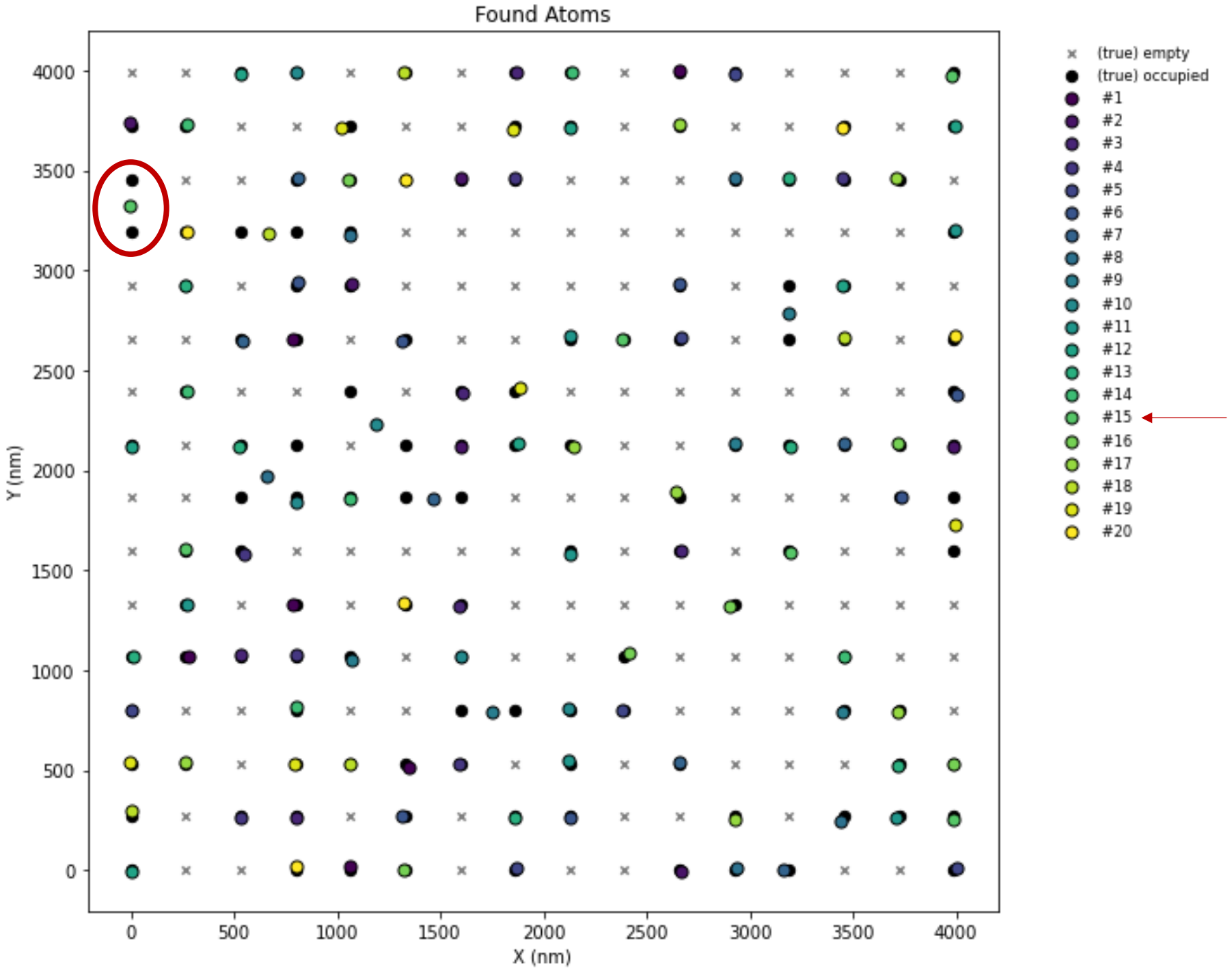
La Rooij, Arthur, et al. "A comparative study of deconvolution techniques for quantum-gas microscope images." *New Journal of Physics* 25.8 (2023): 083036.



How to find peaks?
First guess using local_max, that also counts peaks. Then, gaussian fit (need to review/check/improve).

Further improvements: use amplitude (and shapes) to distinguish single or double atoms fluorescence.

Here I counted 1 atom instead of 2!



The error: summing each “|occupation-truth|”

occupation - NumPy object array

	8	9	10	11	12	13	14
0	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0
3	0	1	1	1	1	1	1
4	0	0	1	0	0	0	1
5	1	1	1	1	0	1	0
6	0	0	1	0	0	1	1
7	1	0	0	2	0	0	0
8	1	0	1	0	0	1	0
9	0	1	0	0	0	1	0
10	0	0	0	0	1	1	1

−

True_occupation - NumPy object array

	8	9	10	11	12	13	14
0	1	1	1	1	1	1	1
1	0	0	0	0	0	1	0
2	1	0	0	0	0	0	1
3	1	1	1	1	1	1	1
4	0	0	1	0	0	0	1
5	1	1	1	1	0	1	0
6	1	0	1	1	0	1	1
7	1	0	1	1	0	0	0
8	1	0	1	0	0	1	0
9	0	1	0	0	0	1	0
10	0	0	0	0	1	1	1

=

=

absdiff - NumPy object array

	8	9	10	11	12	13	14
0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0
2	0	0	0	0	0	0	1
3	1	0	0	0	0	0	0
4	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0
6	1	0	0	1	0	0	0
7	0	0	1	1	0	0	0
8	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0

Our error will be the sum of these boxes

Other possible choices can be done, such as the squared difference... or computing the error in another way

MC-simulation for increasing N_img

Here I'm showing the MC-mean of the (just discussed) error

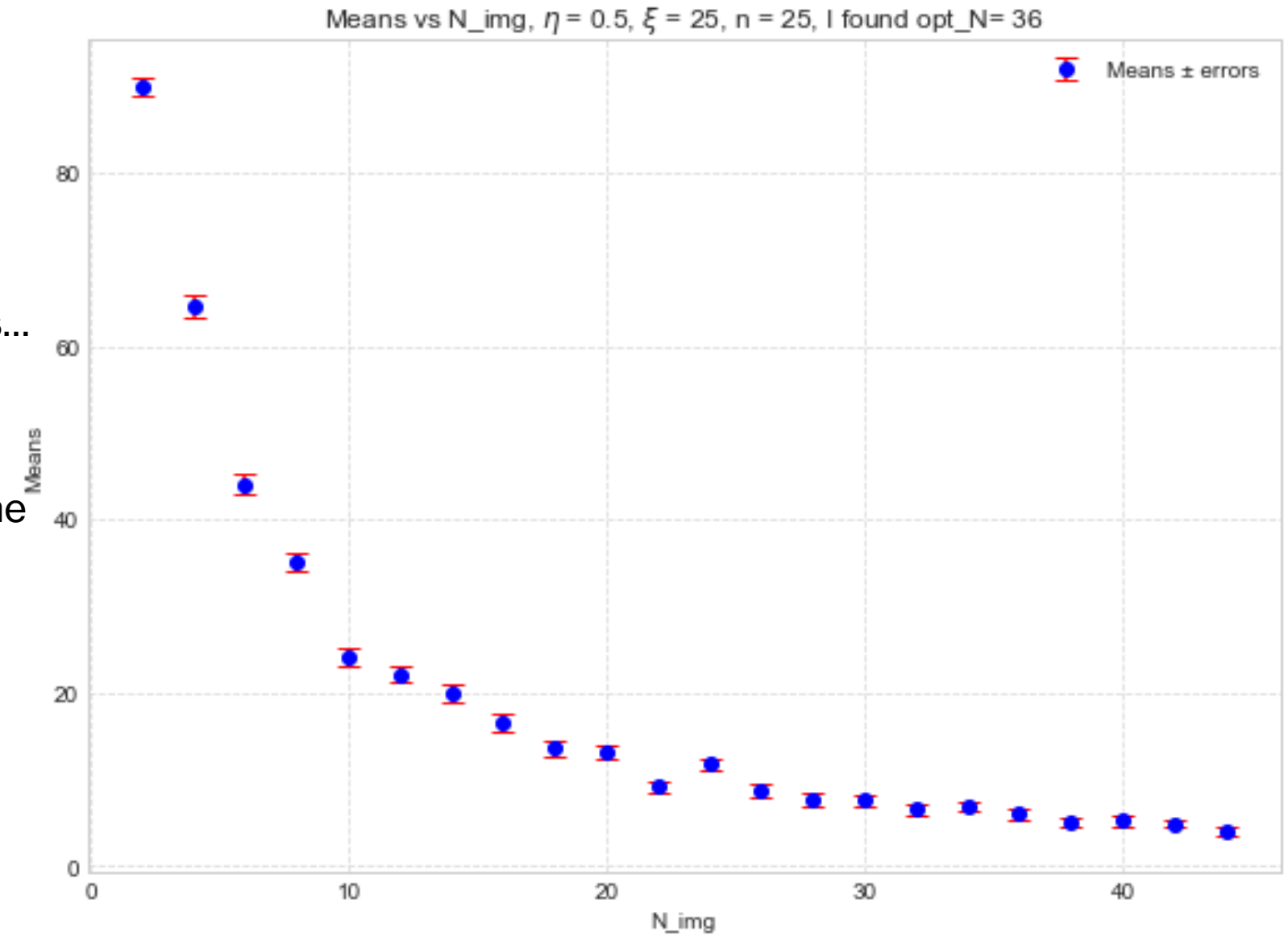
$$err = \sum_{i,j} |Occ_{ij} - Truth_{ij}|$$

in function of the number of the images... looking for the «optimum number»,

where
n = #iterations on which we compute the arithmetical means,
η = filling fraction
ξ = noise-level

(lattices are randomly and uniformly generated)

(Other results are available changing the initial parameters)

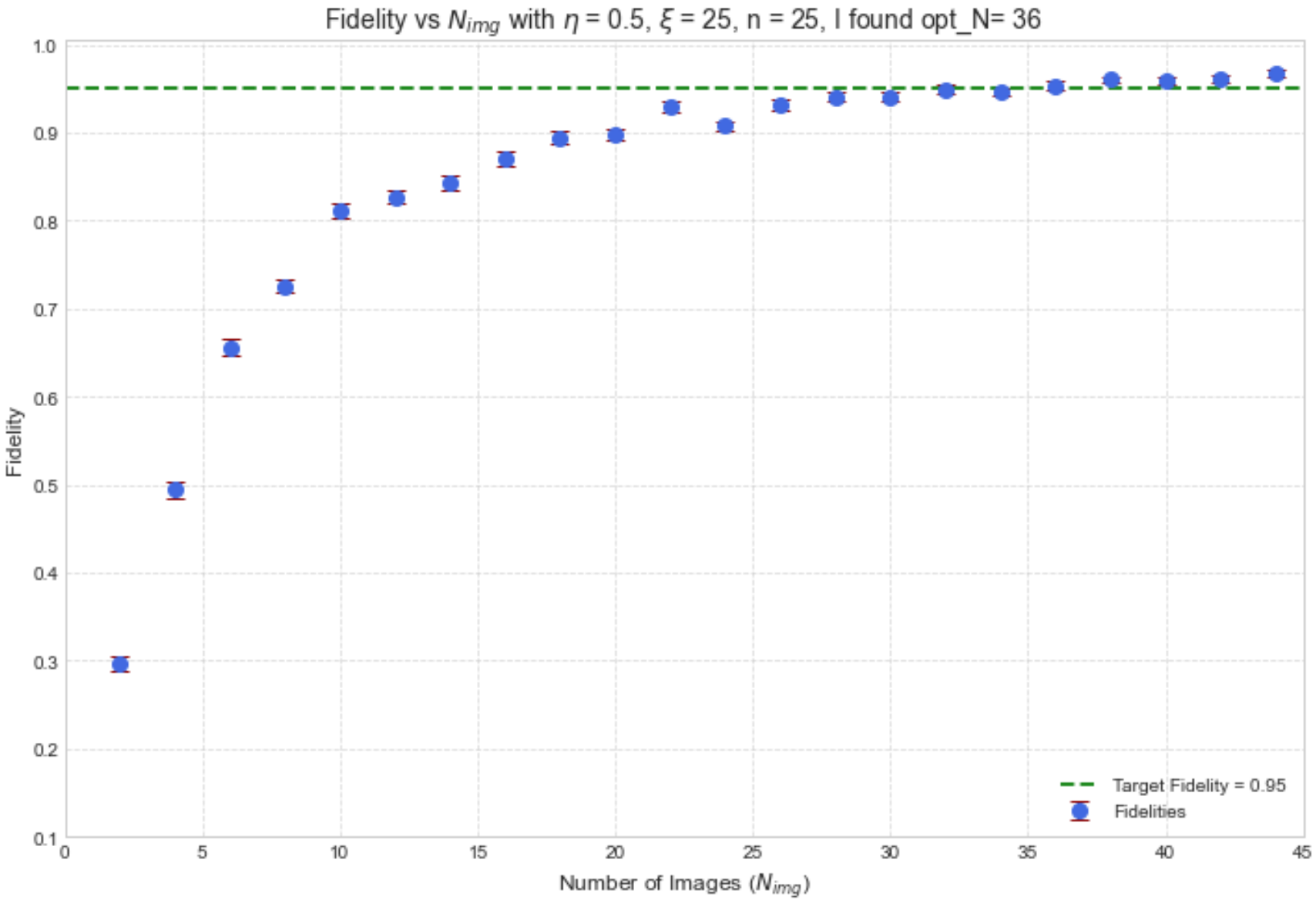


We defined the «optimum number» of images such that the «fidelity», defined as

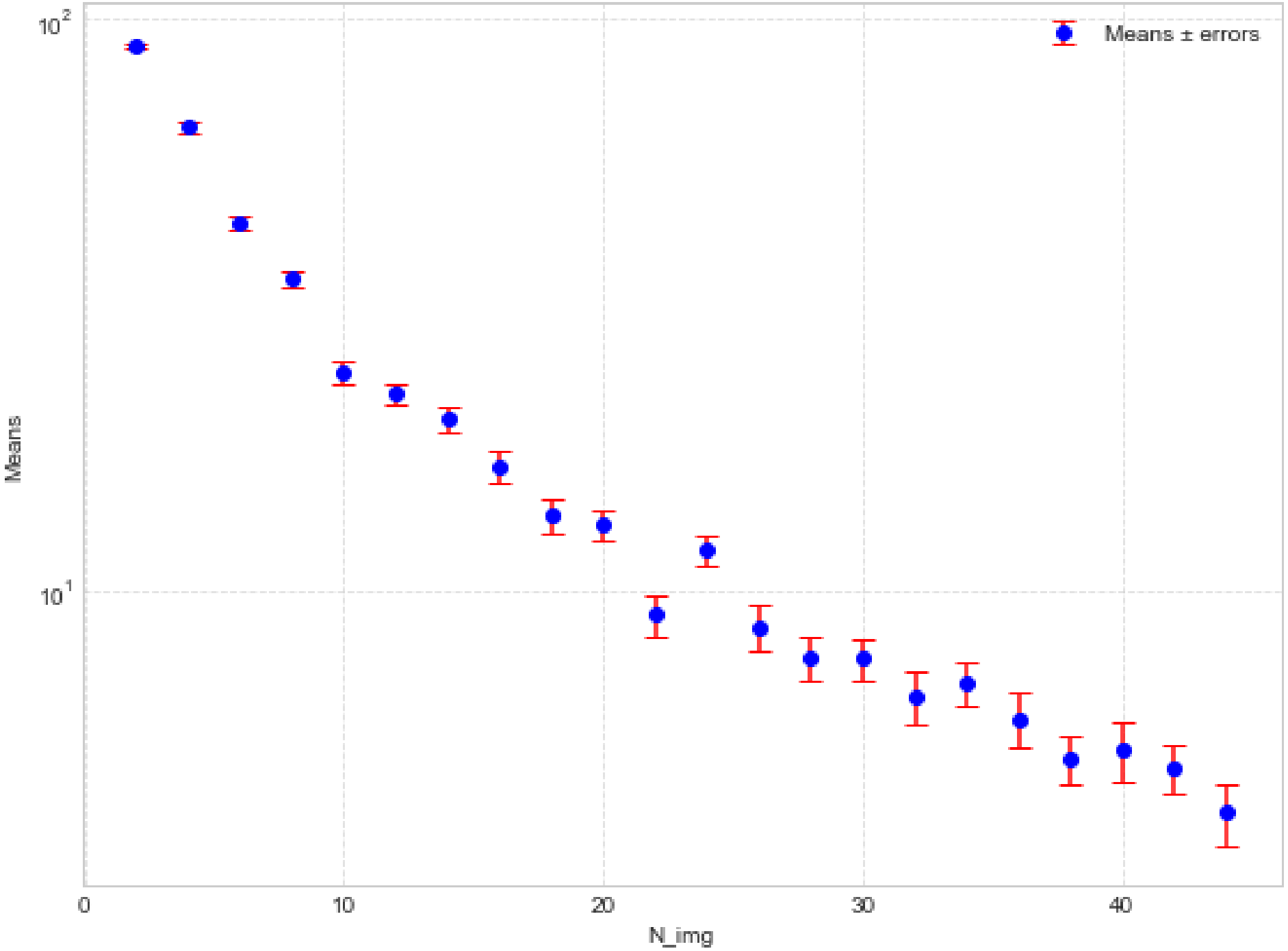
$$F = 1 - \frac{err}{\#atoms},$$

reaches 95%

Observation: it doesn't look exponential! (next slide...)



In fact, the log-plot (on y-axis) of the means (of our error) is not linear...



Conclusions, comments, open questions

- Trial results: 40 images are enough to get a fidelity >95% for η=0.5, ξ=0.25
- In microbiology, however, thousands of images are needed... in our case the lattice is regular and «known» and we have many fewer atoms/cells to resolve!

Requiring the same #atoms per image is a good trade-off? Or start with a few atoms and increase?

Other Deconvolution Methods are available.

Are there other ways to distinguish neighbouring atoms?

Improve the algorithm that finds the peaks' coordinates, or find a better one.

Which is the best choice for our «error»? How will the previous plots change?

Are there more efficient MC-simulations?

Thank you!